

Stateful Workflows in AI Engineering: The Missing Skill Behind Reliable AI Agents

Why production AI agents need memory, checkpoints, retries, human approval, and structured state.

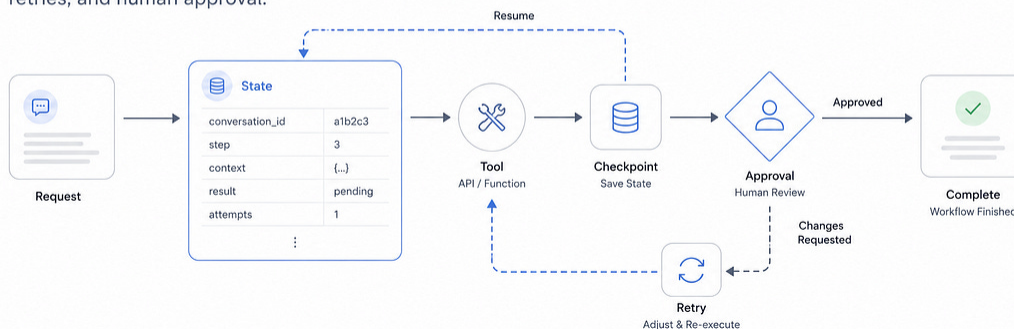


AI ENGINEERING INSIDER

JUN 01, 2026

Stateful Workflows in AI Engineering

Reliable AI agents need memory, checkpoints, retries, and human approval.



★ | MEMORY | CHECKPOINTS | RETRIES | HUMAN IN THE LOOP | RELIABLE BY DESIGN

Most people think an AI workflow is just:

User request → AI response

This Substack is reader-supported. To receive new posts and support my work, consider becoming a free or paid subscriber.

✓ Subscribed

But production AI systems are not that simple.

A real AI agent may need to:

- ↳ Understand the user request
- ↳ Retrieve documents
- ↳ Call APIs
- ↳ Wait for human approval
- ↳ Retry failed steps
- ↳ Resume after a crash
- ↳ Remember what already happened
- ↳ Decide what to do next

That is where stateful workflows become important.

A stateful workflow is one of the most important concepts in AI engineering, especially if you are building agentic AI, LangGraph systems, workflow automation, RAG agents, or human-in-the-loop applications.

What Is a Stateful Workflow?

A stateful workflow is one that remembers information from previous steps and uses that memory to decide what to do next.

In simple words:

A stateless workflow starts from zero every time.

A stateful workflow remembers where it is.

For example, imagine an AI customer support agent.

A stateless system only sees this message:

Can you check my refund?

It may not know who the user is, what order they mean, whether the refund was already submitted, or what happened earlier.

A stateful system remembers:

- ↳ The user is already logged in
- ↳ The order ID was collected earlier
- ↳ The refund request was already submitted
- ↳ The current status is waiting for manager approval
- ↳ The next action should be notifying the user, not asking for the order ID again

That is the power of the state.

The system does not behave like a chatbot that forgets everything.

It behaves like a workflow that knows its progress.

Stateless vs Stateful Workflow

Type	Meaning	Example
Stateless workflow	Each step does not remember previous steps unless the data is passed again	Simple API call
Stateful workflow	The workflow stores memory, progress, and execution context	AI agent handling a multi-step task

A stateless workflow is useful for simple tasks.

For example:

- ↳ Translate this sentence
- ↳ Summarize this paragraph

- ↳ Classify this text
- ↳ Call one API and return the result

But stateful workflows are needed when the task is longer, more complex, or requires decision-making across multiple steps.

For example:

- ↳ AI customer support agent
- ↳ AI travel booking agent
- ↳ AI research assistant
- ↳ AI coding agent
- ↳ AI workflow automation system
- ↳ AI refund approval workflow
- ↳ AI document review pipeline

Why Stateful Workflows Matter in AI Engineering

AI agents are not just prompt templates.

A real AI agent usually needs to keep track of:

- ↳ User request
- ↳ Conversation history
- ↳ Retrieved documents
- ↳ Tool results
- ↳ Current task status
- ↳ Errors and retries
- ↳ Human approval status
- ↳ Next action to execute
- ↳ Final output status

Without a state, the agent becomes unreliable.

It may ask the same question again.

It may call the same tool twice.

It may forget that the user already approved something.

It may lose progress after an error.

It may restart the entire workflow instead of continuing from the failed step.

This is why stateful workflows are foundational for production-grade AI systems.

Example: AI Travel Booking Agent

Imagine a user says:

Book me a trip to New York from June 10 to June 15 with a budget of \$1000.

A stateful AI travel agent may store:

```
{
  "destination": "New York",
  "dates": "June 10-15",
  "budget": "$1000",
  "flight_selected": true,
  "hotel_selected": false,
  "needs_human_approval": true,
  "current_step": "waiting_for_user_approval"
}
```

Now the agent knows:

↳ The destination is already known

↳ The dates are already known

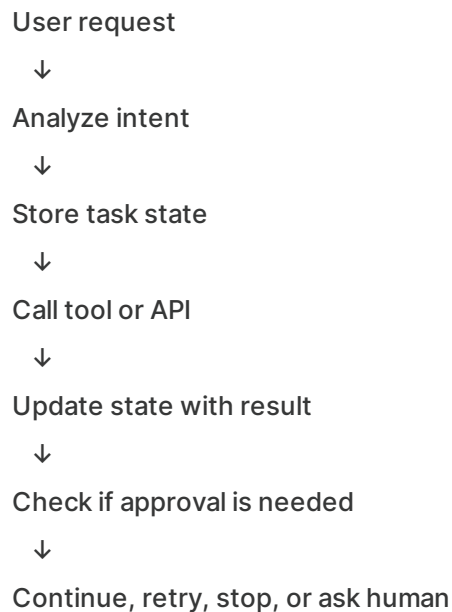
- ↳ The budget is already known
- ↳ The flight is already selected
- ↳ The hotel has not been selected yet
- ↳ User approval is still required

So the next step is not to restart the conversation.

The next step is to continue from the current state.

A Typical Stateful AI Workflow

A stateful AI workflow may look like this:



This is very different from a simple prompt-response system.

In production AI engineering, the model is only one part of the system.

The workflow around the model is what makes the system reliable.

What Does a State Usually Contain?

A workflow state object usually contains operational memory.

Example:

```
{  
  "user_request": "I want a refund for my last order",  
  "intent": "refund_request",  
  "order_id": "ORD-12345",  
  "refund_amount": 250.00,  
  "refund_eligible": true,  
  "approval_required": true,  
  "approval_status": "pending",  
  "current_step": "waiting_for_manager_approval",  
  "retry_count": 0,  
  "final_response": null  
}
```

This state tells the system exactly where the workflow is.

It also helps the system decide what to do next.

Workflow State Is Not the Same as Chat Memory

This is an important interview point.

Many people confuse workflow state with conversation history.

They are related, but they are not the same.

Concept	Meaning	Example
Conversation memory	What the user and AI said	The user asked about a refund
Workflow state	What the system is doing	The current step is waiting for approval
Tool state	Results from external tools	Refund API returned eligible
Business state	Domain process status	Refund is pending manager approval
Long-term memory	Persistent user knowledge	User prefers email notifications

Conversation history helps the AI understand the discussion.

Workflow state controls execution.

That difference matters in production systems.

Why LangGraph Is Important for Stateful AI

In the LangChain ecosystem, LangGraph is commonly used for building stateful, multi-step AI agents.

LangChain is useful for building LLM chains, prompts, tools, retrievers, and model interactions.

LangGraph is useful when your AI system needs:

- ↳ Stateful execution
- ↳ Graph-based workflows
- ↳ Conditional routing
- ↳ Checkpointing
- ↳ Human-in-the-loop approval

- ↳ Multi-step agent control
- ↳ Long-running workflow support

In simple terms:

LangChain helps you build AI components.

LangGraph helps you orchestrate stateful AI workflows.

Code Sample: Stateful Workflow with LangGraph

Below is a simple LangGraph example for a refund workflow.

The workflow remembers:

- ↳ User request
- ↳ Order ID
- ↳ Refund amount
- ↳ Eligibility status
- ↳ Approval status
- ↳ Current step
- ↳ Final response

```
# pip install -U langgraph langchain-core

from typing import TypedDict, Optional, Literal
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver

class RefundState(TypedDict):
    user_request: str
    order_id: Optional[str]
    refund_amount: Optional[float]
    refund_eligible: Optional[bool]
    approval_required: bool
```

```
approval_status: Literal["not_required", "pending", "approved", "rejected"]
current_step: str
final_response: Optional[str]
```

```
def extract_order_info(state: RefundState) → dict:
```

```
    """
```

```
    In production, an LLM or order service could extract this.
```

```
    For demo purposes, we hard-code the result.
```

```
    """
```

```
    return {
        "order_id": "ORD-12345",
        "refund_amount": 250.00,
        "current_step": "order_info_extracted"
    }
```

```
def check_refund_eligibility(state: RefundState) → dict:
```

```
    """
```

```
    In production, this would call an order/refund policy API.
```

```
    """
```

```
    return {
        "refund_eligible": True,
        "current_step": "eligibility_checked"
    }
```

```
def decide_approval(state: RefundState) → dict:
```

```
    """
```

```
    Business rule:
```

```
    Refunds above $100 require manager approval.
```

```
    """
```

```
    amount = state["refund_amount"] or 0
```

```
    if amount > 100:
```

```
        return {
```

```

        "approval_required": True,
        "approval_status": "pending",
        "current_step": "waiting_for_manager_approval"
    }

    return {
        "approval_required": False,
        "approval_status": "not_required",
        "current_step": "approval_not_required"
    }

def approval_router(state: RefundState) → str:
    if state["approval_required"]:
        return "wait_for_approval"
    return "process_refund"

def wait_for_approval(state: RefundState) → dict:
    """
    In production, this could pause and wait for human approval.
    For this demo, we simulate approval.
    """
    return {
        "approval_status": "approved",
        "current_step": "approval_received"
    }

def process_refund(state: RefundState) → dict:
    """
    In production, this would call the Refund API.
    Use an idempotency key to prevent duplicate refunds.
    """
    if state["refund_eligible"] and state["approval_status"] in ["approved",
"not_required"]:
```

```

return {
    "current_step": "refund_processed",
    "final_response": (
        f"Refund for order {state['order_id']} has been approved "
        f"and processed for ${state['refund_amount']}."
    )
}

```

```

return {
    "current_step": "refund_rejected",
    "final_response": "Refund could not be processed."
}

```

```

builder = StateGraph(RefundState)

```

```

builder.add_node("extract_order_info", extract_order_info)
builder.add_node("check_refund_eligibility", check_refund_eligibility)
builder.add_node("decide_approval", decide_approval)
builder.add_node("wait_for_approval", wait_for_approval)
builder.add_node("process_refund", process_refund)

```

```

builder.add_edge(START, "extract_order_info")
builder.add_edge("extract_order_info", "check_refund_eligibility")
builder.add_edge("check_refund_eligibility", "decide_approval")

```

```

builder.add_conditional_edges(
    "decide_approval",
    approval_router,
    {
        "wait_for_approval": "wait_for_approval",
        "process_refund": "process_refund"
    }
)

```

```

builder.add_edge("wait_for_approval", "process_refund")

```

```
builder.add_edge("process_refund", END)

checkpointer = InMemorySaver()
graph = builder.compile(checkpointer=checkpointer)

initial_state: RefundState = {
    "user_request": "I want a refund for my last order.",
    "order_id": None,
    "refund_amount": None,
    "refund_eligible": None,
    "approval_required": False,
    "approval_status": "not_required",
    "current_step": "started",
    "final_response": None
}

config = {
    "configurable": {
        "thread_id": "refund-workflow-001"
    }
}

result = graph.invoke(initial_state, config=config)

print(result["final_response"])
print("Final step:", result["current_step"])
print("Approval status:", result["approval_status"])
```

Expected output:

```
Refund for order ORD-12345 has been approved and processed for $250.0.
Final step: refund_processed
Approval status: approved
```

Why This Code Is Stateful

Each node reads the current state and returns an update.

The workflow moves through these states:

```
started
  ↓
order_info_extracted
  ↓
eligibility_checked
  ↓
waiting_for_manager_approval
  ↓
approval_received
  ↓
refund_processed
```

The workflow does not forget where it is.

It knows the order ID.

It knows the refund amount.

It knows approval is required.

It knows approval was received.

It knows the refund was processed.

That is a stateful workflow.

Where Checkpointing Fits

Checkpointing means saving the workflow state during execution.

If the workflow fails after checking refund eligibility, it should not restart from zero.

It should resume from the last saved state.

Example:

```
Step 1: Extract order info → checkpoint saved
Step 2: Check eligibility → checkpoint saved
Step 3: Wait for approval → checkpoint saved
Step 4: Process refund → failed
Step 5: Retry refund only
```

This is very important in production.

Without checkpointing, failures can cause duplicate work, lost context, repeated user questions, or duplicate API calls.

Human-in-the-Loop Stateful Workflow

Many production AI systems need human approval.

Example:

```
AI drafts refund decision
↓
State updated: approval_required = true
↓
Workflow pauses
↓
Manager approves or rejects
↓
State updated: approval_status = approved
↓
```

Workflow resumes



Customer is notified

This is impossible to do reliably without a state.

The system needs to remember:

- ↳ What is waiting for approval
- ↳ Who needs to approve it
- ↳ What data was reviewed
- ↳ What decision was made
- ↳ Where to resume after approval

LangChain Conversation Memory Example

For simple conversation memory, you may not need a full LangGraph workflow.

You can use LangChain message history.

This is useful when you only need to remember chat messages.

```
# pip install -U langchain-core
```

```
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.messages import HumanMessage, AIMessage
from langchain_core.runnables import RunnableLambda
from langchain_core.runnables.history import RunnableWithMessageHistory
```

```
store = {}
```

```
def get_session_history(session_id: str):
```



```

if session_id not in store:
    store[session_id] = InMemoryChatMessageHistory()
return store[session_id]

def simple_chatbot(messages):
    last_message = messages[-1].content.lower()

    if "my name is" in last_message:
        name = last_message.split("my name is")[-1].strip()
        return AIMessage(content=f"Nice to meet you, {name}.")

    if "what is my name" in last_message:
        return AIMessage(content="Your name is Lamhot.")

    return AIMessage(content="How can I help you?")

chain = RunnableLambda(simple_chatbot)

chain_with_history = RunnableWithMessageHistory(
    chain,
    get_session_history
)

config = {
    "configurable": {
        "session_id": "user-001"
    }
}

response1 = chain_with_history.invoke(
    [HumanMessage(content="Hi, my name is Lamhot")],
    config=config
)

```

```
response2 = chain_with_history.invoke(  
    [HumanMessage(content="What is my name?")],  
    config=config  
)  
  
print(response1.content)  
print(response2.content)
```

This is useful for chat history.

But it is not enough for complex workflows.

For production AI agents, you usually need a structured workflow state.

LangChain vs LangGraph

Use Case	Better Option
Prompt templates	LangChain
Simple LLM chains	LangChain
Tool calling	LangChain or LangGraph
Conversation memory	LangChain message history
Stateful agent workflows	LangGraph
Conditional routing	LangGraph
Human approval	LangGraph
Checkpointing	LangGraph
Long-running tasks	LangGraph
Production agent orchestration	LangGraph

Production-Grade Design Considerations

A good stateful AI workflow should include:

- ↳ Clear state schema
- ↳ State validation
- ↳ Checkpointing
- ↳ Retry policy
- ↳ Idempotency key
- ↳ Human approval status
- ↳ Audit logging
- ↳ Error handling
- ↳ Observability

- ↳ Security controls
- ↳ State versioning

Common Production Risks

Risk	Example	Prevention
Duplicate action	Refund submitted twice	Use idempotency keys
Stale state	Old price used for booking	Refresh critical data
Lost progress	Workflow crashes mid-task	Use checkpointing
Repeated questions	The agent forgets the collected data	Store structured state
Privacy leak	Sensitive data stored in plain text	Encrypt or minimize state
Race condition	Two agents update the same workflow	Use locks or version checks
Bad retry	Same failed API called repeatedly	Add retry limits and backoff
Unclear debugging	Nobody knows where the workflow failed	Add logs and traces

Interview Question: What Is a Stateful Workflow?

A stateful workflow is a workflow that stores execution context across multiple steps. It remembers the current step, previous results, tool outputs, errors, approval status, and next action. This allows the system to continue intelligently rather than starting from scratch every time.

Interview Question: Why Are Stateful Workflows Important for AI Agents?

They are important because AI agents often perform multi-step tasks. An agent may need to retrieve documents, call APIs, wait for human

approval, retry failures, and resume after interruptions. Without state, the agent may lose progress, repeat questions, or make unsafe decisions.

Interview Question: What Should Be Stored in the Workflow State?

A workflow state should store only what is needed to continue the task safely and correctly.

Common fields include:

- ↳ Workflow ID
- ↳ User request
- ↳ Current step
- ↳ Task status
- ↳ Tool results
- ↳ Retrieved documents
- ↳ Error message
- ↳ Retry count
- ↳ Approval status
- ↳ Final response

Interview Question: What Is the Difference Between State and Memory?

Memory usually refers to conversation history or long-term user information.

State refers to the operational progress of a workflow.

For example, remembering that a user likes morning flights is memory.

Remembering that the workflow is currently waiting for payment approval is state.

Interview Question: How Do You Prevent Duplicate Actions?

Use idempotency.

For example, if a refund API call fails and the workflow retries, the system should not submit two refunds.

You can prevent this by using:

- ↳ Idempotency keys
- ↳ Transaction IDs
- ↳ Request deduplication
- ↳ State transition checks
- ↳ External API safeguards

Interview Question: How Would You Design a Stateful AI Refund Agent?

I would design the system with a graph-based workflow. The state would store the user request, customer ID, order ID, refund amount, refund eligibility, approval status, current step, retry count, and final response.

The workflow would classify the request, extract the order ID, check refund eligibility, determine whether manager approval is required, pause for approval if needed, process the refund, and notify the customer.

For production readiness, I would add checkpointing, idempotency keys, retry policies, audit logs, state validation, and monitoring.

This Substack is reader-supported. To receive new posts and support my work, consider becoming a free or paid subscriber.

✓ Subscribed